РАСПРЕДЕЛЁННЫЕ ВЫЧИСЛЕНИЯ

Отчёт по лабораторной работе № 1

Параллельные вычисления с POSIX Threads

Мандельброт, задача N тел и блокировка чтения-записи

Выполнили студенты группы 23КНТ3

Сибатуллин Самир Хамидович

Камалетдинов Тимур Рамильевич

Денисов Александр Алексеевич

Дисциплина: «Распределённые вычисления»

Преподаватель: Пеплин Фёдор Сергеевич

Октябрь 2026

1 Цель работы и вычислительная среда

В работе реализованы и исследованы три параллельные программы на C с POSIX Threads: поиск точек множества Мандельброта, моделирование гравитационного взаимодействия N тел и односвязный список с собственной блокировкой чтения-записи. Цель экспериментов - определить, как независимые вычисления, барьерная синхронизация и общий изменяемый ресурс влияют на производительность при увеличении числа потоков.

На выбранных нагрузках максимальное ускорение составило 9,62 для Мандельброта и 4,41 для основной версии N тел. Список с преобладанием чтения быстрее всего работал на четырёх потоках, а при 20% изменяющих операций обе блокировки показали лучшее время на одном потоке. Далее приведены алгоритмы, полные таблицы замеров и объяснение этих различий.

1.1 Реализованные компоненты

Задача	Основные файлы и результат
Мандельброт	task1_mandelbrot/mandelbrot.c CSV с координатами x, y
N тел	task2_nbody/nbody.c CSV с t, x1, y1, …, xn, yn
Блокировка чтения-записи	task3_rwlock/my_rwlock.c и my_rwlock.h pth_ll_rwl.c, test_rwlock.c

1.2 Аппаратная и программная конфигурация

Параметр	Значение
Процессор хоста	Intel Core Ultra 5 125H, x86-64; 14 физических ядер и 18 логических процессоров
Оперативная память	32 ГБ установлено; в журнале WSL доступно около 15 GiB
Операционная среда	Windows 11; WSL2; Ubuntu 24.04.5 LTS
Ядро Linux	6.18.54.1-microsoft-standard-WSL2
Компилятор	GCC 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Сборка	-O2 -Wall -Wextra -pthread; для N тел -lm; для списка также -Wno-unused-result

WSL сообщает 18 доступных CPU и виртуальную схему 9 ядер по 2 потока. Эта схема не описывает физическую топологию гибридного процессора. Число физических ядер и память хоста уточнены средствами Windows; характеристики среды запуска взяты из журнала эксперимента.

2 Методика измерений

Основной эксперимент выполнен 17 октября 2026 года с 22:08:42 до 22:12:29 MSK. Использован журнал report/results_2026-10-17_22-08-42.txt. Для каждой конфигурации выполнено пять последовательных запусков нового процесса. Всего измерено 49 конфигураций, или 245 запусков. Предыдущие прогоны в эти данные не включены.

Нагрузка	Параметры
Мандельброт	500 000 принятых точек; MAX_ITER = 1000
N тел, две версии	N = 2000; 100 шагов; Δt = 1 с; tend = 100 с; out_every = 0
Список, почти только чтение	1000 начальных ключей; 1 000 000 операций; поиск 99,9%, вставка 0,05%, удаление 0,05%
Список, смешанная нагрузка	1000 начальных ключей; 100 000 операций; поиск 80%, вставка 10%, удаление 10%
Число потоков p	1, 2, 4, 8, 16, 18, 36 для каждой версии и нагрузки

2.1 Границы измеряемого участка

Макрос GET_TIME из timer.h вызывает gettimeofday. Измеряется прошедшее реальное время от начала создания рабочих потоков до завершения всех pthread_join. Таким образом, время включает запуск и завершение потоков, ожидания, синхронизацию и вытеснение планировщиком, но не включает компиляцию и чтение исходных данных.

У Мандельброта запись CSV выполняется после замера; bench.sh направляет результат в /dev/null. В задаче N тел запись при измерениях отключена параметром out_every = 0. В списке начальное заполнение и освобождение памяти выполняются вне замера; итоговое объединение счётчиков потоков входит в измеряемый участок.

2.2 Рассчитываемые показатели

$$T_{min}(p) = \min(t_{p,1}, \ldots, t_{p,5})$$

$$T_{avg}(p) = (t_{p,1} + \ldots + t_{p,5}) / 5$$

$$S(p) = T_{min}(1) / T_{min}(p), \; E(p) = S(p) / p$$

Минимум характеризует лучший наблюдавшийся результат, среднее отражает все пять повторов. Ускорение и эффективность вычисляются по минимумам отдельно для каждой версии программы. Поэтому ускорения двух алгоритмов нельзя сравнивать без сравнения их абсолютного времени. В таблицах время указано в секундах, E - доля от единицы.

Индивидуальные пять значений журнал не сохраняет, поэтому стандартное отклонение и доверительные интервалы не рассчитываются. Специальное закрепление потоков за CPU и отдельный прогрев в скриптах отсутствуют. Эксперимент характеризует запуск в WSL на данном ноутбуке.

3 Поиск точек множества Мандельброта

3.1 Последовательная проверка точки

Для комплексной точки $c = x + iy$ строится последовательность $z_0 = 0$, $z_{k+1} = z_k^2 + c$. Если $|z|^2 > 4$, точка отвергается. Если за 1000 итераций выхода не произошло, программа принимает точку. Это конечное численное приближение к множеству, а не доказательство ограниченности бесконечной последовательности.

Кандидаты выбираются равномерно в прямоугольнике: x от -2 до 0,5, y от -1,25 до 1,25. Поиск с отбраковкой продолжается, пока не будет получено требуемое число принятых точек. Функция in_mandelbrot выполняет итерации с действительными компонентами, проверяя квадрат модуля без вычисления корня.

3.2 Разделение работы между потоками

Для потока r, где $0 \le r < p$, индексы результата образуют полуинтервал [floor(rN/p); floor((r+1)N/p)). Здесь floor означает округление вниз. Поток thread_work заполняет только свой участок массивов xs и ys. Пересечений записей нет, поэтому блокировка массива не нужна. Каждый поток использует своё состояние генератора my_drand с начальным seed = r + 1; главный поток записывает CSV после всех pthread_join.

Память для результата имеет порядок O(N). Стоимость проверки одного кандидата ограничена 1000 итерациями; общее число кандидатов зависит от отбраковки. Равные квоты принятых точек дают близкую, но не строго одинаковую вычислительную нагрузку. При изменении p меняется и набор используемых случайных последовательностей.

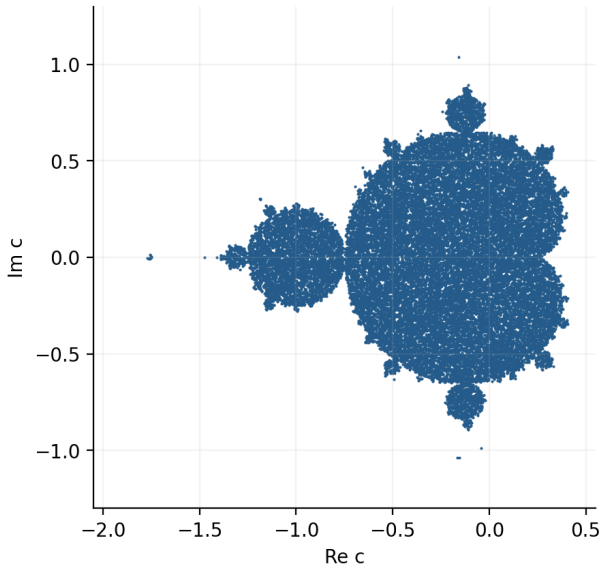


Рисунок 1 - 30 000 точек, полученных программой на четырёх потоках. На осях указаны действительная и мнимая части c.

Демонстрационный CSV содержит ровно 30 000 пар координат. Пример построен отдельно от серии замеров на 500 000 точках. Особенности округления координат у границы рассмотрены в разделе 6.

3.3 Производительность Мандельброта

p	Tmin, c	Tavg, c	S	E
1	1.069164	1.077391	1.00	1.00
2	0.537307	0.541636	1.99	0.99
4	0.278329	0.477539	3.84	0.96
8	0.167058	0.172468	6.40	0.80
16	0.111875	0.115109	9.56	0.60
18	0.112378	0.117154	9.51	0.53
36	0.111132	0.114797	9.62	0.27

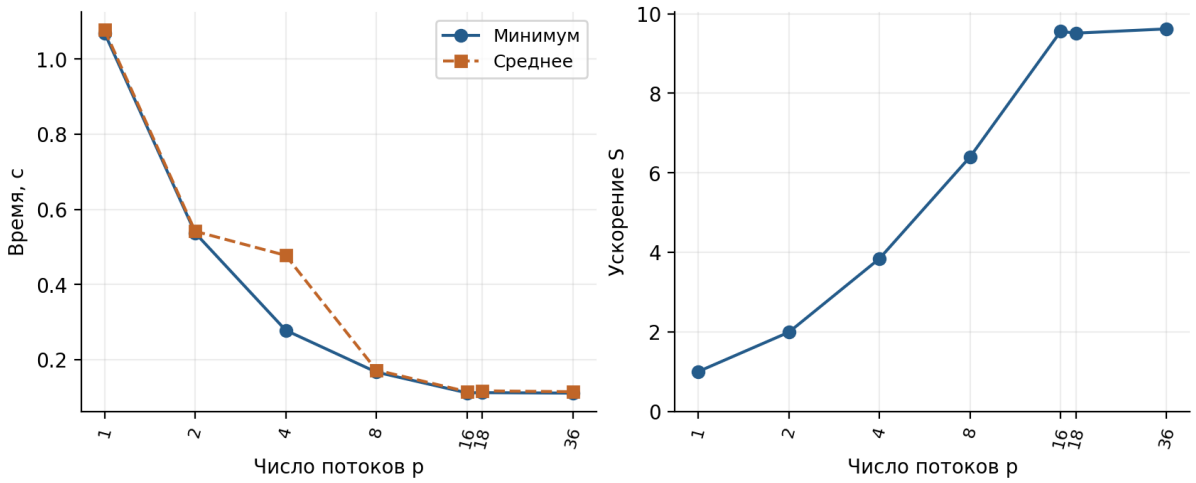


Рисунок 2 - Время поиска 500 000 точек и ускорение по минимальному времени. Каждый результат основан на пяти повторах.

От одного до четырёх потоков ускорение близко к линейному: 1,99 на двух потоках и 3,84 на четырёх. При восьми потоках оно достигает 6,40, при шестнадцати - 9,56. Независимая обработка точек позволяет эффективно использовать параллелизм без регулярных общих блокировок.

После 16 потоков время практически выходит на плато. Лучший минимум 0,111132 с получен при p = 36, что соответствует S = 9,62, но он лишь на 0,66% меньше результата на 16 потоках. По этим данным нельзя уверенно считать 36 потоков устойчиво лучшей конфигурацией. Эффективность при этом уменьшается с 0,60 до 0,27.

Для четырёх потоков среднее 0,477539 с заметно выше минимума 0,278329 с: отношение равно 1,72. Это показывает неоднородность повторов, однако без отдельных времён нельзя установить, сколько запусков были медленнее и чем именно вызвана задержка. Минимумы и средние поэтому приведены одновременно.

Практический выбор для данной задачи - около 16 потоков: это уже область насыщения производительности. Дополнительные потоки не дали существенного уменьшения времени на данной нагрузке.

4 Моделирование гравитационного взаимодействия

4.1 Математическая модель и шаг по времени

Система состоит из N материальных точек с постоянными массами, трёхмерными координатами и скоростями. Сила, действующая на тело q со стороны k, определяется законом всемирного тяготения; силы от остальных тел суммируются. Единицы входных данных: килограммы, метры и секунды.

Параметр nthreads задаёт число рабочих потоков параллельной программы, а dt - шаг Δt метода Эйлера.

$$F_{qk} = G\, m_q m_k (r_k - r_q) / |r_k - r_q|^3$$

$$G = 6{,}67430 \cdot 10^{-11}\ м^3/(кг \cdot с^2)$$

$$F_q = \sum_{k \neq q} F_{qk},\ F_{kq} = -F_{qk}$$

Интегрирование выполняется явным методом Эйлера первого порядка. Координаты обновляются по старой скорости, затем скорость - по силе, рассчитанной из старых координат. Число шагов в коде равно floor(tend/Δt + 0,5); в экспериментах отношение tend/Δt целое.

$$r_q^{new} = r_q^{old} + v_q^{old} \Delta t$$

$$v_q^{new} = v_q^{old} + F_q^{old} \Delta t / m_q$$

4.2 Использование третьего закона Ньютона

Основная версия nbody перебирает только пары q < k. Для каждой пары одна вычисленная сила прибавляется к телу q и вычитается из тела k. Число расчётов взаимодействий на шаг уменьшается с N(N - 1) до N(N - 1)/2. Внешние индексы q распределяются циклически: r, r + p, r + 2p, …, что сглаживает разную длину внутренних циклов.

Чтобы потоки не записывали одновременно в одну силу, каждый накапливает результат в собственном массиве loc_force[r][q]. После барьера reduce_forces суммирует частичные силы. Суммирование и обновление тел распределены по непересекающимся непрерывным блокам.

Этап одного шага	Состояние общих данных
1 Вычисление парных сил	Координаты только читаются; запись идёт в локальные силы потока
Барьер 1	Все частичные силы готовы; чтение старых координат закончено
2 Суммирование и обновление	Поток записывает итоговую силу, позицию и скорость только своих тел
Барьер 2	Все новые координаты готовы для следующего шага

Сложность основного расчёта - $O(N^2)$ на шаг; дополнительные локальные силы занимают $O(pN)$ памяти. Суммарная работа по их обнулению и сведению составляет $O(pN)$ на шаг. Версия nbody_basic из того же файла собирается с -DBASIC и вычисляет оба направления каждой пары; она служит только базой сравнения.

4.3 Производительность задачи N тел

Обе версии используют один входной файл на 2000 тел, 100 шагов и отключённый вывод CSV. Для N = 2000 основная версия вычисляет 1 999 000 пар на шаг, а базовая - 3 998 000 направленных взаимодействий. Время ниже не включает генерацию и чтение входного файла.

Основная версия nbody с третьим законом Ньютона

p	Tmin, c	Tavg, c	S	E
1	0.495647	0.501806	1.00	1.00
2	0.259399	0.261276	1.91	0.96
4	0.140400	0.141843	3.53	0.88
8	0.127022	0.131673	3.90	0.49
16	0.112562	0.115129	4.40	0.28
18	0.112290	0.121624	4.41	0.25
36	0.151562	0.156201	3.27	0.09

Базовая версия nbody_basic с полным перебором

p	Tmin, c	Tavg, c	S	E
1	0.954958	0.961888	1.00	1.00
2	0.500909	0.505929	1.91	0.95
4	0.265080	0.274933	3.60	0.90
8	0.224993	0.229708	4.24	0.53
16	0.190843	0.192126	5.00	0.31
18	0.193384	0.203674	4.94	0.27
36	0.219362	0.223350	4.35	0.12

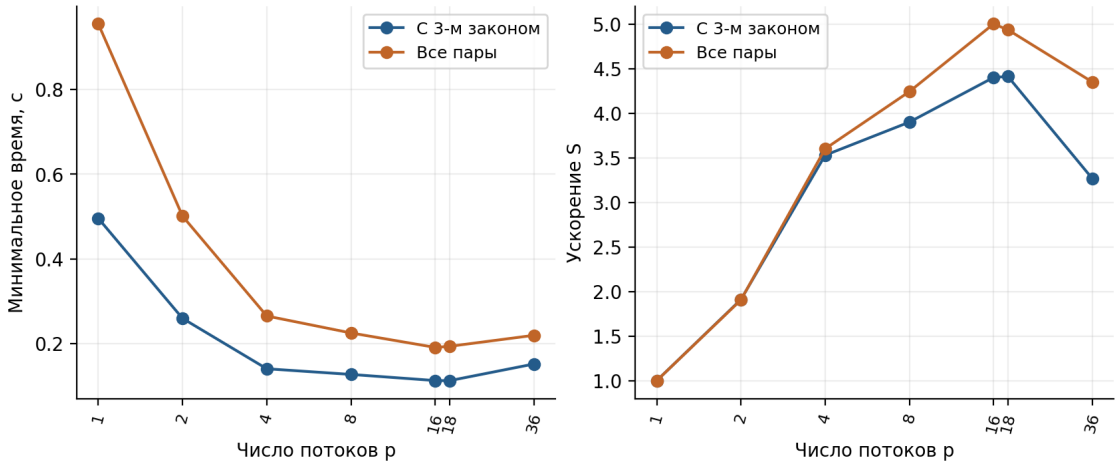


Рисунок 3 - Минимальное время и ускорение двух версий. Каждая кривая ускорения нормирована на свой запуск с одним потоком.

4.4 Траектории и анализ масштабирования

Использование третьего закона уменьшает объём парных вычислений и даёт меньшее абсолютное время на всех измеренных p. На одном потоке выигрыш относительно nbody_basic равен 1,93, на 18 - 1,72. При этом собственное ускорение базовой версии больше: до 5,00 против 4,41. Причина в разных однопоточных базах сравнения; больший S у более медленного алгоритма не означает меньшего времени.

Основная версия насыщается примерно на 16-18 потоках, а при 36 замедляется до 0,151562 с, на 35,0% относительно 18. На каждом шаге есть два барьера, обнуление локальных массивов и сведение сил. Вместе с конкуренцией 36 потоков за 18 доступных CPU это объясняет ограниченное масштабирование. Вклад отдельных причин не измерялся профилировщиком.

Демонстрация на файле solar.txt

Отдельно рассчитана модель из пяти тел: Солнце, Меркурий, Венера, Земля и Марс. Имена сопоставлены массам и начальным расстояниям из solar.txt. Параметры: четыре потока, tend = 31 536 000 с (365 суток), Δt = 3600 с, запись каждые 24 шага. Получено 366 строк состояния, включая t = 0.

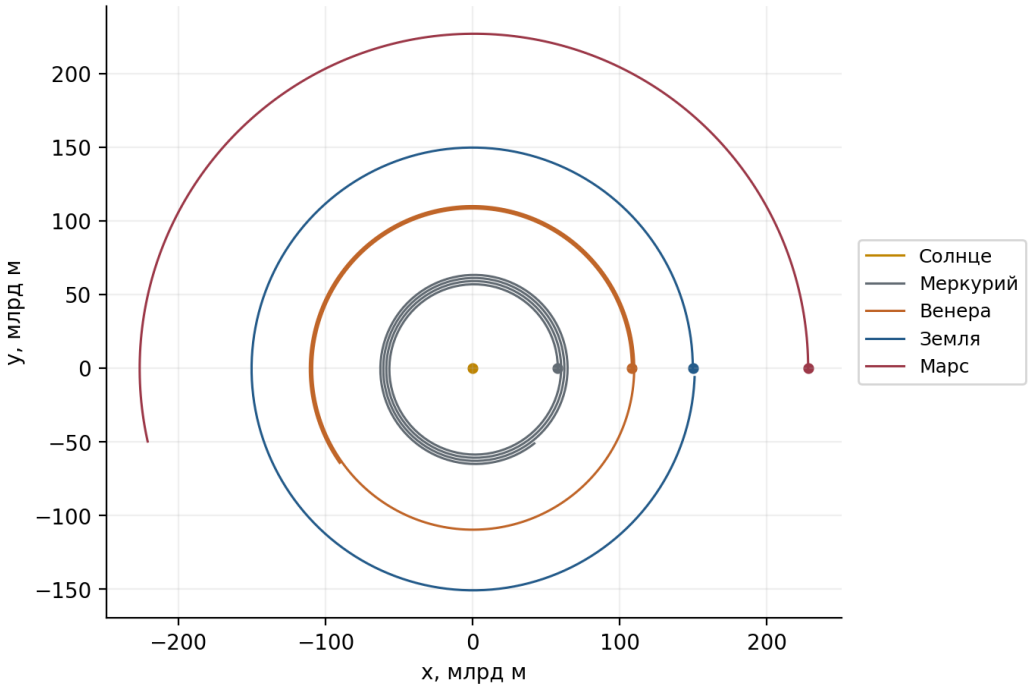


Рисунок 4 - Проекция траекторий пяти тел на плоскость xy за 365 суток. Точки отмечают начальные положения; шаг вывода равен одним суткам.

Расчёт выполняется в трёх измерениях, а CSV содержит t и пары x, y, как требуется заданием. Траектории иллюстрируют работу программы, но не подтверждают астрономическую точность: начальные условия упрощены, а явный метод Эйлера допускает накопление ошибки энергии. Выбор меньшего Δt нужен для исследования сходимости и точности, которое в этот замер производительности не входит.

5 Собственная блокировка чтения-записи

5.1 Состояние и правила доступа

Тип my_rwlock_t содержит внутренний мьютекс, две условные переменные, счётчики активных и ожидающих читателей, счётчик ожидающих писателей и признак активного писателя. Мьютекс защищает только состояние блокировки. После получения разрешения операция со списком выполняется без удержания этого внутреннего мьютекса.

Функция	Условие ожидания и изменение состояния
my_rwlock_rdlock	Увеличить readers_waiting. Ожидать, пока есть активный писатель или ожидающие писатели. После допуска уменьшить readers_waiting и увеличить readers_active.
my_rwlock_wrlock	Увеличить writers_waiting. Ожидать, пока есть писатель или readers_active > 0. После допуска уменьшить writers_waiting и установить writer_active = 1.
my_rwlock_unlock	Снять writer_active или уменьшить readers_active. Когда читателей не осталось, разбудить одного ожидающего писателя; если писателей нет - всех ожидающих читателей.

Все проверки и изменения состояния выполняются под одним мьютексом. pthread_cond_wait атомарно отпускает его на время ожидания и снова захватывает перед возвратом. Условие проверяется в цикле while: после пробуждения право входа мог получить другой поток, также возможны ложные пробуждения.

5.2 Инварианты и приоритет

Число активных писателей не превышает одного. Если писатель активен, активных читателей нет. Несколько читателей могут работать одновременно. Новые читатели не проходят перед уже ожидающим писателем; это политика приоритета писателей. Она не задаёт строгую очередь FIFO и не гарантирует ограниченное время ожидания каждого потока. При непрерывном потоке писателей читатели могут ожидать долго.

5.3 Применение к односвязному списку

Список хранит ключи в возрастающем порядке. Member использует блокировку чтения, Insert и Delete - блокировку записи. Блокируется вся операция обхода и изменения списка. Благодаря этому читатель не может обратиться к узлу, который в тот же момент удаляется другим потоком.

Из одного pth_ll_rwl.c собираются ll_lib с pthread_rwlock_t и ll_my с my_rwlock_t через -DUSE_MY_RWLOCK. Алгоритмы списка и генерация операций совпадают; меняется реализация блокировки. Потоки накапливают локальные счётчики операций и один раз объединяют их под отдельным count_mutex.

Операции выбираются псевдослучайно, seed потока равен r + 1. Проценты задают вероятности выбора операций, а не фиксируют их точное число. Порядок успешных вставок и удалений зависит от планирования потоков.

5.4 Список с преобладанием поиска

Начальный список содержит 1000 ключей. Задано 1 000 000 операций: вероятность поиска 0,999, вставки 0,0005 и удаления 0,0005. Для каждой строки каждой реализации выполнено пять повторов.

p	Библиотечная Tmin, с	Библиотечная Tavg, с	Собственная Tmin, с	Собственная Tavg, с
1	0.707498	0.714233	0.705472	0.710551
2	0.387802	0.582723	0.423971	0.426187
4	0.284290	0.287279	0.385684	0.397718
8	0.378662	0.394961	0.519277	0.539175
16	0.505274	0.514474	0.644837	0.654155
18	0.571063	0.576630	0.676168	0.690695
36	0.877652	0.897327	0.834041	1.032619

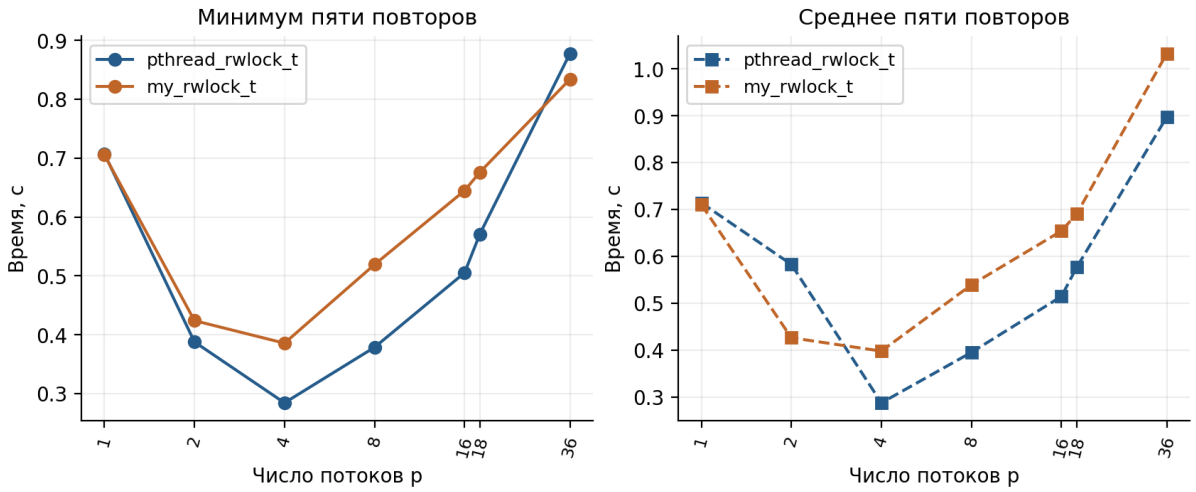


Рисунок 5 - Время работы списка с преобладанием поиска.

Обе реализации достигают своего лучшего минимума на четырёх потоках. Библиотечная блокировка даёт 0,284290 с и ускорение 2,49; собственная - 0,385684 с и ускорение 1,83. На этой конфигурации собственная реализация медленнее библиотечной в 1,36 раза по минимуму.

После четырёх потоков время растёт. Чтения допускают совместный доступ, но каждый вход и выход затрагивает состояние общей блокировки. Также сохраняются обход связанного списка и конкуренция за память. У собственной реализации счётчики дополнительно проходят через один внутренний мьютекс. Эксперимент не разделяет стоимость этих факторов.

При p = 36 минимум ll_my немного меньше минимума ll_lib (0,834041 против 0,877652 с), но среднее, наоборот, больше (1,032619 против 0,897327 с). Поэтому общего вывода о превосходстве собственной блокировки здесь нет. Для ll_lib при p = 2 отношение среднего к минимуму равно 1,50.

5.5 Список со смешанной нагрузкой

Начальный список содержит 1000 ключей. Задано 100 000 операций с вероятностями 0,8 для поиска, 0,1 для вставки и 0,1 для удаления. Число операций меньше, чем в предыдущем режиме; абсолютное время между режимами напрямую не нормируется.

p	Библиотечная Tmin, с	Библиотечная Tavg, с	Собственная Tmin, с	Собственная Tavg, с
1	1.046524	1.059487	1.069047	1.078979
2	1.229302	1.237972	1.094122	1.099521
4	1.263911	1.269874	1.321224	1.536290
8	1.433207	1.465156	1.963399	1.981496
16	1.887903	1.919459	2.837556	2.881348
18	2.113528	2.433657	2.987713	3.252250
36	2.999696	3.056236	4.887785	5.160789

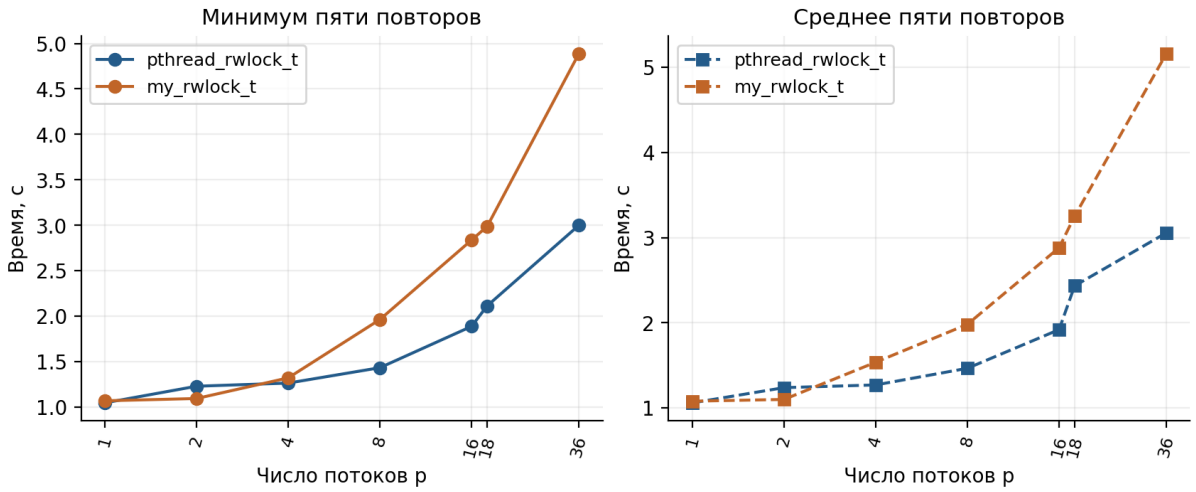


Рисунок 6 - Время работы списка с 20% изменяющих операций.

Лучшее время обеих версий получено на одном потоке: 1,046524 с для ll_lib и 1,069047 с для ll_my. Увеличение p не дало ускорения относительно собственной однопоточной базы. На 36 потоках библиотечная версия замедляется в 2,87 раза, собственная - в 4,57 раза.

Вставка и удаление требуют исключительного доступа, поэтому конкурирующие операции чаще ожидают блокировку. Кроме того, ключи выбираются из диапазона до 100 000 000: случайная вставка часто добавляет новый узел, а случайное удаление редко находит существующий. Размер списка в процессе меняется, а стоимость его обхода может увеличиваться. Это нагрузка на список в целом, а не изолированный тест одного вызова rwlock.

На двух потоках собственная версия быстрее библиотечной по минимуму (1,094122 против 1,229302 с). Начиная с четырёх, её время выше. Наблюдения показывают зависимость результата от смеси операций и числа потоков; универсального выигрыша собственной реализации не обнаружено.

6 Проверка корректности

Функциональные проверки выполнены отдельно от серии измерений. Они проверяют форматы результата и отдельные свойства программ; их времена не добавлялись в таблицы производительности.

6.1 Мандельброт

Для четырёх потоков и 30 000 точек проверены заголовок x,y, число строк и координаты в заданной области. Дополнительная проверка использовала непосредственно исходную функцию in_mandelbrot и воспроизвела исходные double-координаты при тех же seed. Все 30 000 исходных точек проходят критерий 1000 итераций.

После чтения округлённых координат из CSV тот же критерий не проходят 9 точек из 30 000 (0,03%). Максимальное изменение одной координаты при записи - около $5 \cdot 10^{-13}$. Это чувствительность точек у границы к округлению: в CSV используются 12 знаков после запятой. Для повторной проверки без такой потери точности формат вывода можно расширить до 17 значащих цифр.

6.2 N тел

Для 32 тел, полученных gen_bodies.py с seed = 1, выполнено 100 шагов по 1000 с с записью каждые 10 шагов. Сравнивались nbody на 1, 4 и 18 потоках и nbody_basic на 1 и 4 потоках. В сохранённых x, y расхождений не обнаружено при точности CSV 11 значащих цифр. Это согласованность результатов на данном примере, а не равенство всех внутренних double-значений.

Для единственного тела сила равна нулю. При начальных x = 10 м, y = -20 м, vx = 3 м/с, vy = -4 м/с программа на четырёх потоках дала x(t) = 10 + 3t и y(t) = -20 - 4t на всём интервале 0-100 с с шагом 1 с. Также проверен формат демонстрации solar.txt: 366 состояний и 11 столбцов.

6.3 Собственная блокировка

test_rwlock использует массив из 256 элементов. Писатель увеличивает все элементы, читатель проверяет их равенство. Дополнительные счётчики фиксируют пересечения читателей и писателей, а число записей сверяется с итоговыми значениями массива. Доли выбора операций: 80% чтение, 20% запись.

Потоки	Операций на поток	Записей: счётчик / под блокировкой	Нарушений	Максимум читателей
4	20 000	16 145 / 16 145	0	3
18	10 000	36 292 / 36 292	0	3

Оба запуска завершились статусом OK, элементы массива согласованы с числом записей. Одновременная работа трёх читателей подтверждает допуск совместного чтения. Конечные прогоны не доказывают отсутствие ошибок при всех возможных расписаниях и не проверяют ограниченность ожидания.

7 Ограничения и общие результаты

7.1 Как следует интерпретировать измерения

Результаты относятся к одному ноутбуку и одной среде WSL2. Влияние фоновой нагрузки Windows, планировщика, неодинаковых ядер, частоты и состояния питания не разделялось. При 36 потоках их число вдвое превышает доступные логические CPU. Поэтому уменьшение эффективности нельзя приписывать одному фактору без дополнительных измерений.

У Мандельброта фиксировано число принятых точек, но не число проверенных кандидатов. У списка зафиксированы вероятности операций, но не их точные последовательности для всех p. Сравнение конфигураций включает эти особенности генерации нагрузки.

В списке каждому потоку назначается total_ops / p операций с целочисленным делением. Фактическое число равно p · floor(total_ops/p). Для 1 000 000 операций это 999 990 при p = 18 и 999 972 при p = 36; для 100 000 - 99 990 и 99 972. Максимальное отклонение от задания в этих опытах составляет 0,028% и не меняет наблюдаемого роста времени.

Задача N тел использует точечные массы без обработки столкновений и без смягчения силы при малом расстоянии. Совпадение координат приводит к делению на ноль. Явный метод Эйлера имеет первый порядок по времени; сохранение энергии и сходимость по Δt в этой работе отдельно не измерялись.

7.2 Сводка лучших наблюдавшихся конфигураций

Программа и нагрузка	p	Tmin, с	S
Мандельброт, 500 000 точек	36	0.111132	9.62
N тел, третий закон Ньютона	18	0.112290	4.41
N тел, полный перебор	16	0.190843	5.00
ll_lib, 99,9% поиска	4	0.284290	2.49
ll_my, 99,9% поиска	4	0.385684	1.83
ll_lib, 80% поиска	1	1.046524	1.00
ll_my, 80% поиска	1	1.069047	1.00

Для Мандельброта разница между 16 и 36 потоками меньше 1%, для основной N-body между 16 и 18 - меньше 0,3%. Таблица фиксирует наблюдавшийся минимум, а не статистически доказанный оптимум. Ускорение S относится к однопоточной базе той же строки программы.

7.3 Выводы

Независимые вычисления Мандельброта масштабируются лучше остальных задач. Третий закон Ньютона почти вдвое сокращает однопоточное время N-body, однако барьеры и сведение сил ограничивают дальнейший выигрыш. Совместное чтение списка полезно при малой доле изменений; при росте числа писателей исключительный доступ и стоимость обхода подавляют преимущество параллелизма. Собственная блокировка выдержала проведённые проверки, но не дала устойчивого преимущества перед pthread_rwlock_t.

8 Воспроизведение эксперимента

Перейдите в корень клонированного или распакованного репозитория distributed-computing-lab1. Все команды и относительные пути ниже отсчитываются от этого каталога. Зависимости устанавливает install_dependencies.sh; Makefile собирают программы с pthreads, Python и Matplotlib используются для генерации входа и графиков.

8.1 Полная серия измерений

```
bash install_dependencies.sh --check
RUNS=5 bash run_benchmarks.sh
```

Скрипт сам собирает задачи, выбирает 1, 2, 4, 8, 16, 18 и 36 потоков при nproc = 18 и сохраняет характеристики системы и таблицы в results_<дата>_<время>.txt. Для другой системы список потоков зависит от числа доступных CPU.

8.2 Примеры запуска программ

```
make

./lab1_pthreads/task1_mandelbrot/mandelbrot 4 30000 points.csv

./lab1_pthreads/task2_nbody/nbody 4 31536000 \
  lab1_pthreads/task2_nbody/solar.txt 3600 24 solar.csv

./lab1_pthreads/task3_rwlock/test_rwlock 4 20000
./lab1_pthreads/task3_rwlock/test_rwlock 18 10000

# 1000 ключей, 100000 операций, 80% поиска и 10% вставок
printf '1000\n100000\n0.8\n0.1\n' | \
  ./lab1_pthreads/task3_rwlock/ll_my 4
```